
Assignment 1

Software Design

Author: Virag Raul

Group: 30433

1 Problem Definition

This project implements a Warehouse Management System - a web application that allows warehouse staff to manage product types, track physical stock units, and perform inventory operations such as adding and searching items.

The system maintains two levels of inventory: a product catalogue (Items) and individual physical units (StockUnits), each identified by a system-generated serial number. A denormalized Stock counter provides fast aggregate quantity of queries.

This document covers the analysis, design, implementation, and testing phases of the project.

2 Analysis Phase — Use Cases & Requirements

2.1 Functional Requirements

- The system must allow users to add, edit, and delete product types (Items).
- The system must allow users to add stock units in bulk; each unit receives a system-generated serial number.
- The system must allow users to search stock units by product name or serial number.
- The system must mark individual units as Available, Reserved, Shipped, or Returned.

2.2 Use Cases

UC-1 — Add Product Type

- Actor: Warehouse Manager + Seller
- Task: Creates a new Item with name, description, and base price.
- Result: Item persisted; visible in product list.

UC-2 — Add Stock Units

- Actor: Warehouse Manager + Seller
- Task: Selects a product type and enters quantity; system creates many StockUnit rows.
- Result: Stock counter incremented; units searchable by serial.

UC-3 — Search Stock

- Actor: Warehouse Staff + Viewer
- Task: Enters a product name or partial serial number in the search bar.
- Result: Filtered list of matching StockUnits displayed.

UC-4 — Delete operation -> only Admin

Uc-5 — Update operation both admin and seller

3 Design Phase

3. Architecture Overview

The application follows a three-layer architecture: Presentation (ASP.NET MVC Razor views), Business Logic (Service classes), and Data Access (Entity Framework Core with SQL Server). The client browser communicates exclusively with the MVC controller layer over HTTP; no direct database access is exposed.

This style isolates concerns: views hold no query logic; services hold no HTML, and repositories (via EF DbContext) are injectable and testable.

Entity-Relationship Diagram

The database contains three core tables:

- **Items** — product catalogue. Attributes: Id, Name, Description, PricePerItem, IsDeleted, DeletedAtTime, LastModifiedTime.
- **StockUnits** — one row per physical unit. Attributes: Id, ItemId (FK), SerialNumber, ActualPrice, Note, Status, IsDeleted.
- **Stocks** — denormalized quantity counter. Attributes: Id, ItemId (FK), Quantity ≥ 0 .

Items → StockUnits: one-to-many. Items → Stocks: one-to-one.

Component: MVC Web Application

The ASP.NET MVC application acts as the sole entry point. Controllers receive HTTP requests, delegate to service classes, and return Razor views. A shared generic partial view (`_ViewGenericTable`) renders tabular data for any entity, keeping view code DRY.

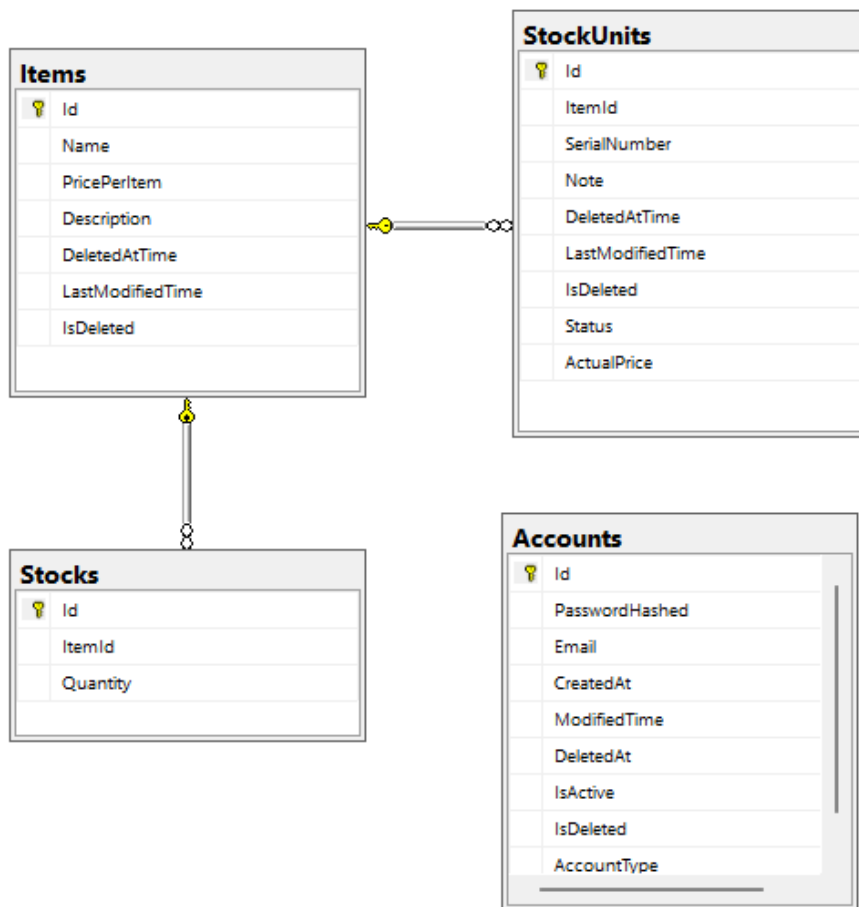
Component: Service & Data Layer

`GenericService<TEntity, TResponse, TCreate, TUpdate>` provides default CRUD operations via EF Core. Concrete services (`ItemService`, `StockUnitService`) override methods that require navigation-property projections or business-rule validation + implement additional methods.

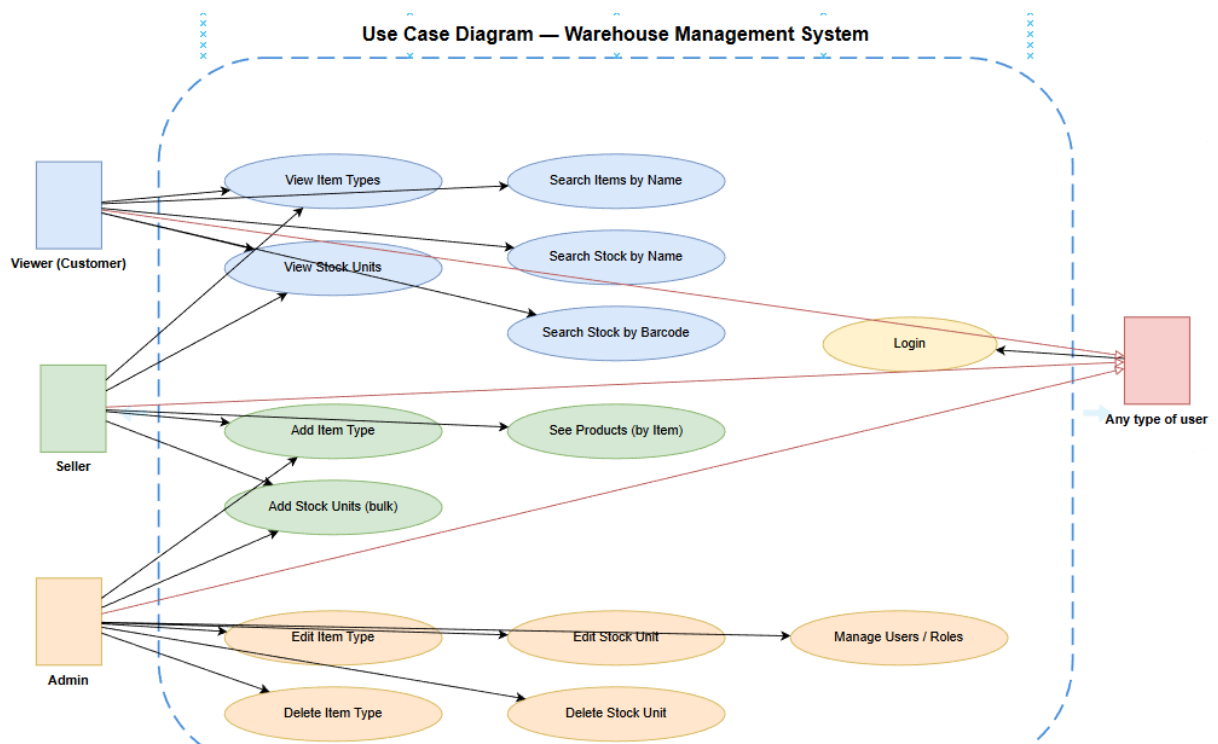
Design Patterns

- **Repository / Unit of Work (structural)** — EF DbContext acts as the unit of work; `DbSet<T>` acts as repository. Decouples business logic from SQL.
- **Generic Service (behavioral)** — a single base class provides CRUD for any entity; subclasses override only what differs.
- **DTO / Mapper (structural)** — AutoMapper translates between EF entities and lightweight DTOs, keeping the view layer decoupled from the data model.

Entity-Relationship Diagram — Warehouse System



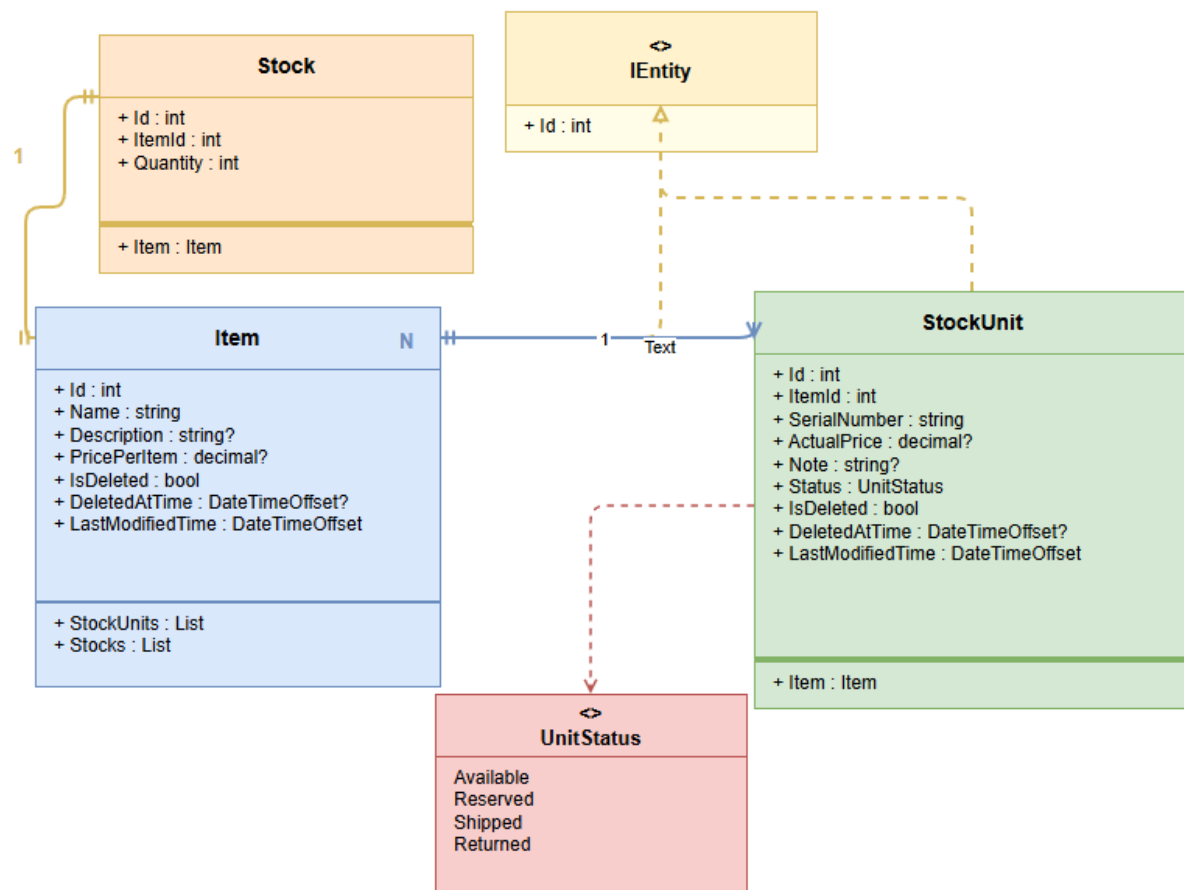
Use case Diagram



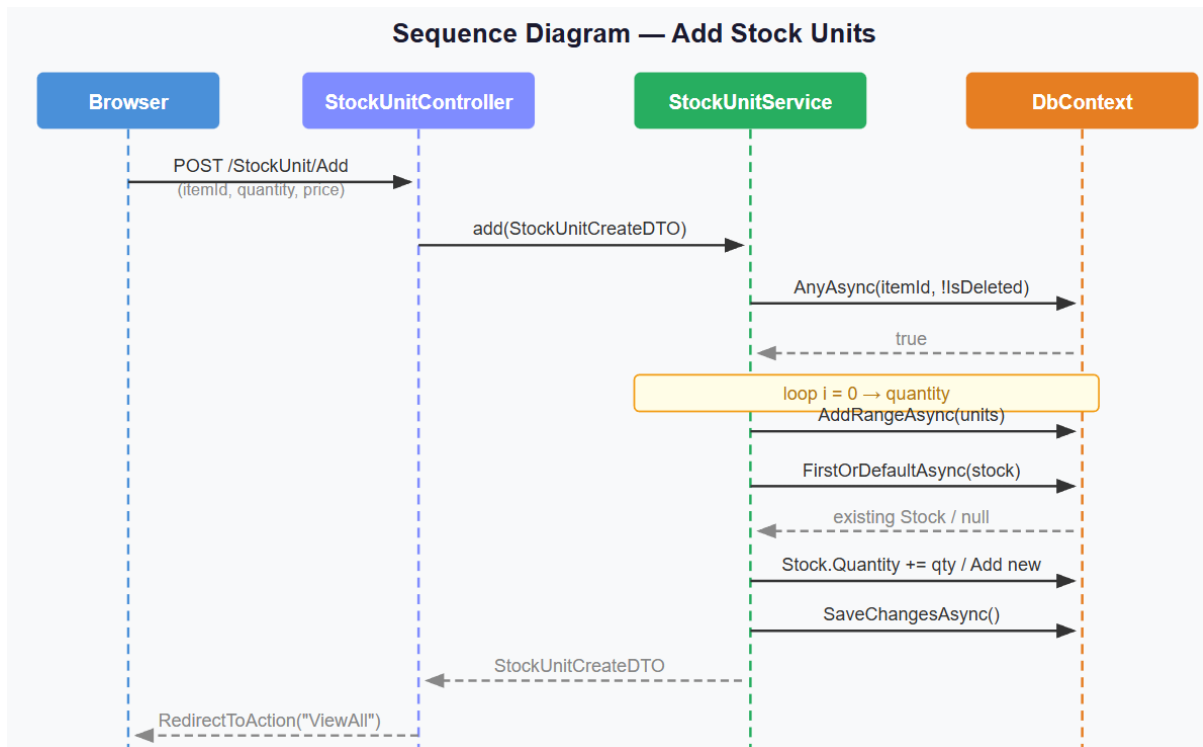
Core Domain Diagram

The entities and their relationships (Item, Stock Unit, Stock, Enums) . It shows the structure of data in the app.

Core Domain Class Diagram — Warehouse System



Sequence Diagram



4 Implementation Phase

4.1 Tools and Technologies

- **C# / .NET 10** — primary language and runtime.
- **ASP.NET Core MVC** — web framework providing routing, model binding, and Razor rendering.
- **Entity Framework Core** — ORM with code-first migrations and LINQ query translation.
- **SQL Server** — relational database; chosen for CHECK constraint support and EF Core compatibility.
- **AutoMapper** — object-to-object mapping between entities and DTOs.
- **Bootstrap 5** — CSS framework for responsive UI.

4.2 Application Structure

- **Warehouse.Data** — Entities, DTOs, DbContext, Migrations.
- **Warehouse.Services** — GenericService base + ItemService, StockUnitService.
- **Warehouse (MVC)** — Controllers, Views (shared partial _ViewGenericTable), AutoMapper profiles.

4.3 User Manual

Navigation bar provides links to Items and Stock Units. Key workflows:

- Items → View All: lists all product types with Edit/Delete and a 'View Stock' link per row.
- Items → Add: form to create a new product type.

- Stock Units → View All: lists all physical units with name and serial. Search bar accepts both product name and partial serial.
- Stock Units → Add: select a product type and enter quantity; system generates serials automatically.

5 Testing and Validation

Validation was performed: manual integration tests via the UI.

5.1 Manual Integration Tests

- Adding 10 stock units generates 10 unique serials and increments quantity.
- Searching by partial serial returns matching units only.